Architecture Concept for the @Work Robot (Multi-Container Setup)

by Tim Lachmann

2026

Introduction: Advantages of a Multi-Container ROS2 Setup

The overall system is made up of four separate Docker containers. They communicate with each other over the ROS2 network using the host network mode.

While ROS2 inherently promotes modularity through its distributed node architecture, encapsulating these major subsystems into isolated Docker containers provides significant advantages for both development and deployment:

- **Eliminating "Dependency Hell":** Complex robotic systems often require conflicting library versions. Containers isolate these environments, ensuring each module has exactly what it needs without interfering with the host OS.
- **Independent Sub-Team Development:** Teams focusing on distinct areas (e.g., Navigation, Vision) can build, test, and run their containers completely independently.
- **Enhanced Modularity & Strict Boundaries:** ROS2 provides process-level modularity, but Docker adds OS-level isolation, enforcing strict architectural boundaries via defined ROS2 interfaces.
- **Guaranteed Reproducibility:** Containerization resolves the "it works on my machine" problem, ensuring identical execution on the developer's laptop, in CI, and on the physical robot.

Internal Container Architecture:
The fundamental conceptual division of this system is heavily inspired by the master's thesis of Marco Masannek [1]. While the original work proposes a holistic layered architecture (Brain, Control, Model, Driver), this concept has been adapted to fit our distributed Docker setup. The overarching "Brain" acts as its own container, while the sub-layers are now implemented *inside* the respective functional containers (Navigation, Vision, Manipulation) to maintain a strict separation of concerns:

- **Control Layer:** Handles the high-level execution, status monitoring, and coordination of the specific module.
- **Model Layer:** Contains the algorithms, mathematical models, and data processing (e.g., SLAM, Path Planning, Neural Networks).
- **Driver Layer:** Manages the direct hardware interfaces and low-level sensor/actuator communication.

1. The Brain Container (`brain_container`)

This is the central brain and main orchestrator of the robot. It decides what needs to be done, but it doesn't execute any physical actions itself.

- **Planner:** The strategic layer. It takes high-level goals, optimizes the route (similar to a Traveling Salesman Problem), and figures out the order of tasks.
- **Executioner (State Machine / Behavior Tree):** The tactical layer. It breaks the plan down into atomic ROS2 calls and triggers the subsystems one after another.
- **World Model:** The knowledge base. It remembers static locations (like the coordinates of points A and B) and dynamic states (like where object X is right now).

2. Navigation Container (`nav_container`)

This module handles the autonomy for the mobile base (e.g., using Nav2).

- It includes Lidar drivers, SLAM/localization for mapping, and path planning (both global and local).
- **Task:** To drive the robot safely and autonomously from its current position to the target coordinates without crashing into anything.

3. Vision Container (`vision_container`)

This is the perception module, hooked up to the camera on the robot arm.

- **Task 1:** Object detection. It finds the 3D coordinates of a specific part we are looking for.
- **Task 2:** Free space detection. When asked, it searches for an empty spot to place an object.
- It only provides data and doesn't make any decisions on its own.

4. Manipulation Container (`manipulation_container`)

This module controls the robot arm and the gripper (e.g., based on MoveIt2).

- It manages the **inventory** (keeping track of up to 3 objects in storage).
  "Comment: Should inventory be stored in the Brain/World Model?"
- It calculates kinematics (how to position the joints) to reach specific 3D coordinates.
- It executes complex macro-actions, like moving into a search position, grabbing an object, or placing it down.

Interface Overview

The system architecture and its communication flow are visualized in the figure below.

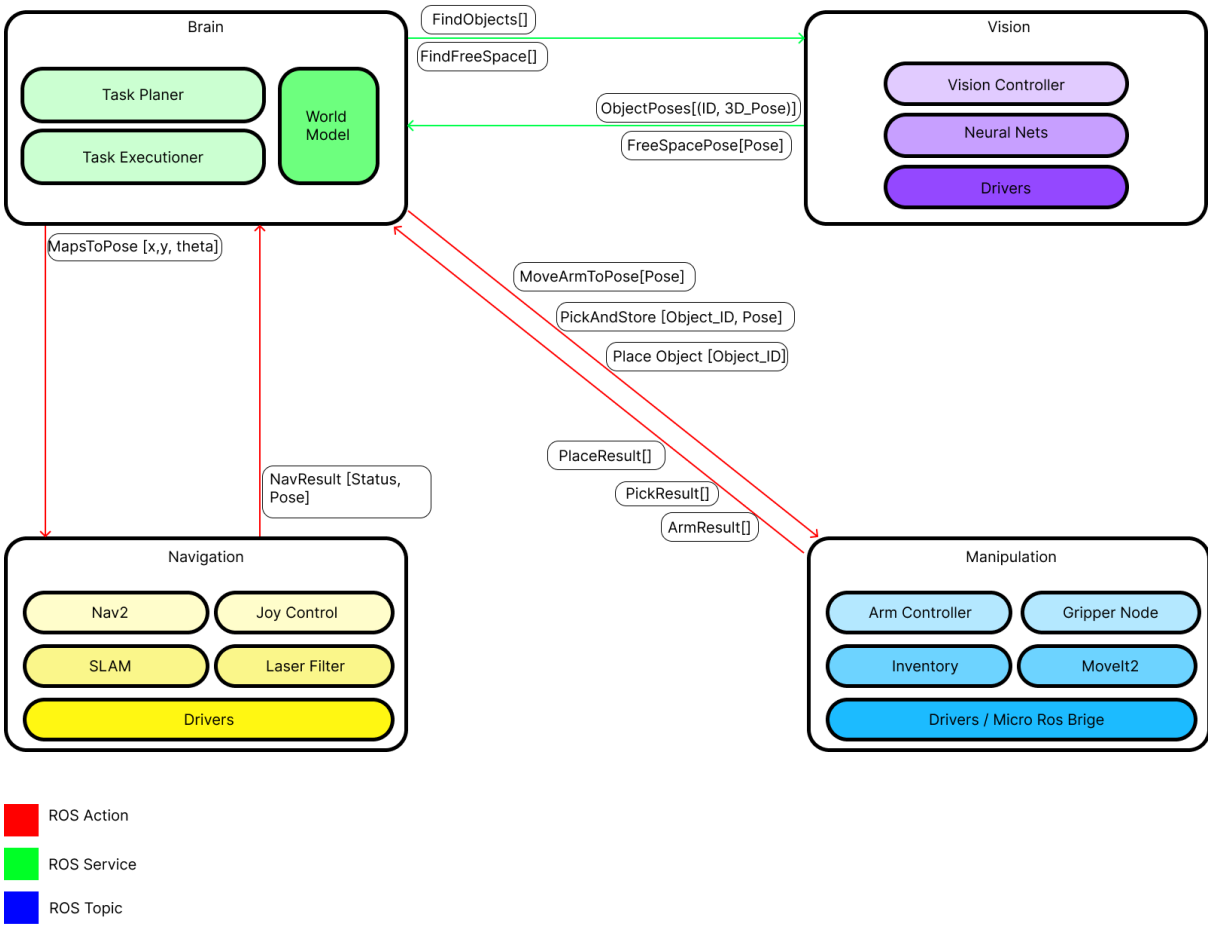

Figure 1: System Interfaces and Communication

Since we are using ROS2, the table below includes the specific communication types (Topic, Service, Action):

- **Actions:** Used for tasks that take a while and need continuous feedback (like driving or grabbing).
- **Services:** Used for quick requests that get an immediate answer (like "Where is a free spot?").

Table 1: Interface Definitions

Sender	**Comm. Type**	**Interface / Payload**	**Receiver**	**Description**
User/System	ROS Action	`TaskGoal` [Object_ID, Start_Loc, Target_Loc]	**Brain (Planner)**	New task for the robot.
Brain (Exec)	ROS Action	`MapsToPose` [x, y, theta]	**Navigation**	Drive to coordinates of location A or B.
Navigation	ROS Action Feedback	`NavResult` [Status: Arrived / Failed]	**Brain (Exec)**	Feedback that the destination was reached.
Brain (Exec)	ROS Action	`MoveArmToState` [State: "Search_Pose"]	**Manipulator**	Move arm to camera position for searching.
Manipulator	ROS Action Feedback	`ArmResult` [Status: Success]	**Brain (Exec)**	Arm has reached the search position.
Brain (Exec)	ROS Service	`FindObject` [Object_ID]	**Vision**	Search for part X in the camera image.
Vision	ROS Serv. Response	`ObjectPose` [Found: Boolean, x, y, z]	**Brain (Exec)**	3D coordinates of part X.
Brain (Exec)	ROS Action	`PickAndStore` [Object_ID, x, y, z]	**Manipulator**	Grab object at (x,y,z) and store it in inventory.
Manipulator	ROS Action Feedback	`PickResult` [Status: Success, Inv_Slots_Used]	**Brain (Exec)**	Object successfully grabbed and stored.
Brain (Exec)	ROS Action	`PlaceObject` [Object_ID]	**Manipulator**	Take an object from inventory and place it down.

Table 2: Interface Definitions

Brain (Exec)	ROS Service	`FindFreeSpace` [BoundingBox / Size]	**Vision**	Peer-to-Peer request: Where is free space to place it?
Vision	ROS Serv. Response	`FreeSpacePose` [x, y, z]	**Brain (Exec)**	3D coordinates of an empty surface.
Manipulator	ROS Action Feedback	`PlaceResult` [Status: Success, Inv_Slots_Used]	**Brain (Exec)**	Part successfully placed down.

References

[1] Marco Masannek, *Implementierung einer Roboterkontrollarchitektur für autonome mobile Roboter im industriellen Umfeld*. Masterarbeit, Fachgebiet Robotik, 2021.